

Namespace PhotoCatalog.Application.

Caching

Classes

[CacheKeysFactory](#)

Централизованная фабрика строковых идентификаторов для кэширования.
Разделяет генерацию Key и Tag.

Class CacheKeysFactory

Namespace: [PhotoCatalog.Application.Caching](#)

Assembly: PhotoCatalog.Application.dll

Централизованная фабрика строковых идентификаторов для кэширования.
Разделяет генерацию Key и Tag.

```
public static class CacheKeysFactory
```

Inheritance

[object](#)  ← CacheKeysFactory

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  ,
[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  ,
[object.ToString\(\)](#) 

Methods

GetFolderAlbumsKey(int)

Ключ для списка альбомов конкретной папки.

```
public static string GetFolderAlbumsKey(int folderId)
```

Parameters

folderId [int](#) 

Идентификатор папки.

Returns

[string](#) 

Уникальный ключ вида `key:folder:{folderId}:albums-key`.

GetFolderTag(int)

Тег, связанный с конкретной папкой. Сброс кэша по этому тегу инвалидирует все данные, относящиеся к папке.

```
public static string GetFolderTag(int folderId)
```

Parameters

folderId [int](#)

Идентификатор папки.

Returns

[string](#)

Тег вида `tag:folder:{folderId}:folder-tag`.

GetFoldersTreeKey()

Ключ для полного дерева папок.

```
public static string GetFoldersTreeKey()
```

Returns

[string](#)

Ключ вида `key:folders-tree-key`.

GetFoldersTreeTag()

Тег для всего дерева папок. Сброс по этому тегу вызывает полную перестройку дерева при следующем запросе.

```
public static string GetFoldersTreeTag()
```

Returns

[string](#)

Тег вида `tag:folders-tree-tag`.

Namespace PhotoCatalog.Application. DTOs

Classes

[AlbumResponse](#)

DTO record для ответа с альбомом.

[CreateFolderRequest](#)

DTO record для запроса создания папки.

[FolderResponse](#)

DTO record для ответа с папкой.

[ImportPhotoRequest](#)

DTO record для запроса импорта фото.

[PhotoResponse](#)

DTO record для ответа с фото.

Class AlbumResponse

Namespace: [PhotoCatalog.Application.DTOs](#)

Assembly: PhotoCatalog.Application.dll

DTO record для ответа с альбомом.

```
public record AlbumResponse : IEquatable<AlbumResponse>
```

Inheritance

[object](#)  ← AlbumResponse

Implements

[IEquatable](#)  <[AlbumResponse](#)>

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  ,
[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  ,
[object.ToString\(\)](#) 

Extension Methods

[ResultExtensions.ToResult<TValue>\(TValue\)](#) ,
[ResultExtensions.ToResult<TValue>\(TValue?, Error\)](#)

Constructors

AlbumResponse(int, string, int?,
IReadOnlyCollection<int>)

DTO record для ответа с альбомом.

```
public AlbumResponse(int Id, string Name, int? FolderId, IReadOnlyCollection<int>  
    PhotoIds)
```

Parameters

Id [int](#) 

Идентификатор альбома.

Name [string](#)

Название альбома.

FolderId [int](#)?

Идентификатор папки, в которой он находится.

PhotoIds [ICollection](#) <[int](#)>

Список из идентификаторов фото, доступный только для чтения.

Properties

FolderId

Идентификатор папки, в которой он находится.

```
public int? FolderId { get; init; }
```

Property Value

[int](#)?

Id

Идентификатор альбома.

```
public int Id { get; init; }
```

Property Value

[int](#)

Name

Название альбома.

```
public string Name { get; init; }
```

Property Value

[string](#)

PhotoIds

Список из идентификаторов фото, доступный только для чтения.

```
public IReadOnlyCollection<int> PhotoIds { get; init; }
```

Property Value

[IReadOnlyCollection](#) <[int](#)>

Class CreateFolderRequest

Namespace: [PhotoCatalog.Application.DTOs](#)

Assembly: PhotoCatalog.Application.dll

DTO record для запроса создания папки.

```
public record CreateFolderRequest : IEquatable<CreateFolderRequest>
```

Inheritance

[object](#) ← CreateFolderRequest

Implements

[IEquatable](#) <[CreateFolderRequest](#)>

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Extension Methods

[ResultExtensions.ToResult<TValue>\(TValue\)](#),
[ResultExtensions.ToResult<TValue>\(TValue?, Error\)](#)

Constructors

CreateFolderRequest(string, int?)

DTO record для запроса создания папки.

```
public CreateFolderRequest(string Name, int? ParentFolderId)
```

Parameters

Name [string](#)

Название папки.

ParentFolderId [int](#)?

Идентификатор родительской папки.

Properties

Name

Название папки.

```
public string Name { get; init; }
```

Property Value

[string](#)

ParentFolderId

Идентификатор родительской папки.

```
public int? ParentFolderId { get; init; }
```

Property Value

[int](#)?

Class FolderResponse

Namespace: [PhotoCatalog.Application.DTOs](#)

Assembly: PhotoCatalog.Application.dll

DTO record для ответа с папкой.

```
public record FolderResponse : IEquatable<FolderResponse>
```

Inheritance

[object](#)  ← FolderResponse

Implements

[IEquatable](#)  <[FolderResponse](#)>

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  ,
[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  ,
[object.ToString\(\)](#) 

Extension Methods

[ResultExtensions.ToResult<TValue>\(TValue\)](#) ,
[ResultExtensions.ToResult<TValue>\(TValue?, Error\)](#)

Constructors

FolderResponse(int, string, int?)

DTO record для ответа с папкой.

```
public FolderResponse(int Id, string Name, int? ParentFolderId)
```

Parameters

Id [int](#) 

Идентификатор папки.

Name [string](#) 

Название папки.

ParentFolderId [int](#)?

Идентификатор родительской папки.

Properties

Id

Идентификатор папки.

```
public int Id { get; init; }
```

Property Value

[int](#)

Name

Название папки.

```
public string Name { get; init; }
```

Property Value

[string](#)

ParentFolderId

Идентификатор родительской папки.

```
public int? ParentFolderId { get; init; }
```

Property Value

[int](#)?

Class ImportPhotoRequest

Namespace: [PhotoCatalog.Application.DTOs](#)

Assembly: PhotoCatalog.Application.dll

DTO record для запроса импорта фото.

```
public record ImportPhotoRequest : IEquatable<ImportPhotoRequest>
```

Inheritance

[object](#)  ← ImportPhotoRequest

Implements

[IEquatable](#)  <[ImportPhotoRequest](#)>

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  ,
[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  ,
[object.ToString\(\)](#) 

Extension Methods

[ResultExtensions.ToResult<TValue>\(TValue\)](#) ,
[ResultExtensions.ToResult<TValue>\(TValue?, Error\)](#)

Constructors

ImportPhotoRequest(string)

DTO record для запроса импорта фото.

```
public ImportPhotoRequest(string SourcePath)
```

Parameters

SourcePath [string](#) 

Путь к файлу.

Properties

SourcePath

Путь к файлу.

```
public string SourcePath { get; init; }
```

Property Value

[string](#)

Class PhotoResponse

Namespace: [PhotoCatalog.Application.DTOs](#)

Assembly: PhotoCatalog.Application.dll

DTO record для ответа с фото.

```
public record PhotoResponse : IEquatable<PhotoResponse>
```

Inheritance

[object](#)  ← PhotoResponse

Implements

[IEquatable](#)  <[PhotoResponse](#)>

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  ,
[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  ,
[object.ToString\(\)](#) 

Extension Methods

[ResultExtensions.ToResult<TValue>\(TValue\)](#) ,
[ResultExtensions.ToResult<TValue>\(TValue?, Error\)](#)

Constructors

PhotoResponse(int, string, string?, int, int, DateTime, IReadOnlyCollection<int>)

DTO record для ответа с фото.

```
public PhotoResponse(int Id, string RealPath, string? FileHash, int Width, int Height, DateTime AddedAt, IReadOnlyCollection<int> TagIds)
```

Parameters

Id [int](#) 

Идентификатор фото.

RealPath [string](#)

Путь к файлу в файловой системе.

FileHash [string](#)

Хеш-сумма файла.

Width [int](#)

Ширина фото.

Height [int](#)

Высота фото.

AddedAt [DateTime](#)

Дата добавления.

TagIds [ICollection](#) <[int](#)>

Список из идентификаторов тегов, доступный только для чтения.

Properties

AddedAt

Дата добавления.

```
public DateTime AddedAt { get; init; }
```

Property Value

[DateTime](#)

FileHash

Хеш-сумма файла.

```
public string? FileHash { get; init; }
```

Property Value

[string](#)

Height

Высота фото.

```
public int Height { get; init; }
```

Property Value

[int](#)

Id

Идентификатор фото.

```
public int Id { get; init; }
```

Property Value

[int](#)

RealPath

Путь к файлу в файловой системе.

```
public string RealPath { get; init; }
```

Property Value

[string](#)

TagIds

Список из идентификаторов тегов, доступный только для чтения.

```
public IReadOnlyCollection<int> TagIds { get; init; }
```

Property Value

[IReadOnlyCollection](#) <[int](#)>

Width

Ширина фото.

```
public int Width { get; init; }
```

Property Value

[int](#)

Namespace PhotoCatalog.Application. Errors

Classes

[ApplicationErrors](#)

Единый статический класс (реестр), который содержит все ошибки уровня Application в виде заранее определенных структур [Error](#).

[ApplicationErrors.Albums](#)

Ошибка, связанная с обновлением альбома

[ApplicationErrors.Files](#)

Ошибки, связанные с файлами.

[ApplicationErrors.Folders](#)

Ошибки, связанные с папками.

[ApplicationErrors.General](#)

Общие ошибки приложения.

[ApplicationErrors.Transactions](#)

Ошибки, связанные с транзакциями.

[ApplicationErrors.UseCases](#)

Ошибки, связанные с выполнением use-cases (сценариев приложения).

Class ApplicationErrors

Namespace: [PhotoCatalog.Application.Errors](#)

Assembly: PhotoCatalog.Application.dll

Единый статический класс (реестр), который содержит все ошибки уровня Application в виде заранее определенных структур [Error](#).

```
public static class ApplicationErrors
```

Inheritance

[object](#)  ← ApplicationErrors

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  ,
[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  ,
[object.ToString\(\)](#) 

Class ApplicationErrors.Albums

Namespace: [PhotoCatalog.Application.Errors](#)

Assembly: PhotoCatalog.Application.dll

Ошибка, связанная с обновлением альбома

```
public static class ApplicationErrors.Albums
```

Inheritance

[object](#)  ← ApplicationErrors.Albums

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  ,
[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  ,
[object.ToString\(\)](#) 

Fields

UpdateFailed

Ошибка, которая обозначает, что при обновлении что то пошло не так

```
public static readonly Error UpdateFailed
```

Field Value

[Error](#)

Class ApplicationErrors.Files

Namespace: [PhotoCatalog.Application.Errors](#)

Assembly: PhotoCatalog.Application.dll

Ошибки, связанные с файлами.

```
public static class ApplicationErrors.Files
```

Inheritance

[object](#)  ← ApplicationErrors.Files

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  ,
[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  ,
[object.ToString\(\)](#) 

Fields

FileNotFoundException

Ошибка, когда физический файл по указанному пути не найден.

```
public static readonly Error FileNotFoundException
```

Field Value

[Error](#)

OrphanedFile

ошибка, когда файл остается осиротевшим на диске

```
public static readonly Error OrphanedFile
```

Field Value

[Error](#)

Class ApplicationErrors.Folders

Namespace: [PhotoCatalog.Application.Errors](#)

Assembly: PhotoCatalog.Application.dll

Ошибки, связанные с папками.

```
public static class ApplicationErrors.Folders
```

Inheritance

[object](#)  ← ApplicationErrors.Folders

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  ,
[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  ,
[object.ToString\(\)](#) 

Fields

CycleDetected

Ошибка, когда обнаружена циклическая зависимость: нельзя переместить папку внутрь её собственного потомка.

```
public static readonly Error CycleDetected
```

Field Value

[Error](#)

Class ApplicationErrors.General

Namespace: [PhotoCatalog.Application.Errors](#)

Assembly: PhotoCatalog.Application.dll

Общие ошибки приложения.

```
public static class ApplicationErrors.General
```

Inheritance

[object](#)  ← ApplicationErrors.General

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  ,
[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  ,
[object.ToString\(\)](#) 

Fields

NotFound

Ошибка, когда запрашиваемая сущность не найдена.

```
public static readonly Error NotFound
```

Field Value

[Error](#)

Class ApplicationErrors.Transactions

Namespace: [PhotoCatalog.Application.Errors](#)

Assembly: PhotoCatalog.Application.dll

Ошибки, связанные с транзакциями.

```
public static class ApplicationErrors.Transactions
```

Inheritance

[object](#)  ← ApplicationErrors.Transactions

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  ,
[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  ,
[object.ToString\(\)](#) 

Fields

CommitFailed

Ошибка, возникающая при попытке зафиксировать (commit) транзакцию.

```
public static readonly Error CommitFailed
```

Field Value

[Error](#)

StartTransactions

Ошибка, которая обозначает, что при начале транзакции что то пошло не так

```
public static readonly Error StartTransactions
```

Field Value

[Error](#)

Class ApplicationErrors.UseCases

Namespace: [PhotoCatalog.Application.Errors](#)

Assembly: PhotoCatalog.Application.dll

Ошибки, связанные с выполнением use-cases (сценариев приложения).

```
public static class ApplicationErrors.UseCases
```

Inheritance

[object](#)  ← ApplicationErrors.UseCases

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  ,
[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  ,
[object.ToString\(\)](#) 

Fields

SystemFailure

Ошибка, возникающая при непредвиденном системном сбое внутри use-case.

```
public static readonly Error SystemFailure
```

Field Value

[Error](#)

Namespace PhotoCatalog.Application.Use Cases

Classes

[AddPhotoToAlbumUseCase](#)

Сценарий использования для добавления фотографии в существующий альбом.

[ImportPhotoUseCase](#)

Содержит сценарий оркестрации добавления нового фото в каталог.

[MoveFolderUseCase](#)

Сценарий использования для перемещения папки.

Class AddPhotoToAlbumUseCase

Namespace: [PhotoCatalog.Application.UseCases](#)

Assembly: PhotoCatalog.Application.dll

Сценарий использования для добавления фотографии в существующий альбом.

```
public class AddPhotoToAlbumUseCase
```

Inheritance

[object](#)  ← AddPhotoToAlbumUseCase

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  ,
[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  ,
[object.ToString\(\)](#) 

Extension Methods

[ResultExtensions.ToResult<TValue>\(TValue\)](#) ,
[ResultExtensions.ToResult<TValue>\(TValue?, Error\)](#)

Constructors

AddPhotoToAlbumUseCase(IAlbumRepository,
IPhotoRepository, IUnitOfWork, ILogger)

```
public AddPhotoToAlbumUseCase(IAlbumRepository albumRepository, IPhotoRepository  
photoRepository, IUnitOfWork unitOfWork, ILogger logger)
```

Parameters

albumRepository [IAlbumRepository](#)

photoRepository [IPhotoRepository](#)

unitOfWork [IUnitOfWork](#)

logger ILogger

Methods

Execute(int, int)

Выполняет добавление фотографии в альбом с сохранением изменений в рамках транзакции.

```
public ResultVoid Execute(int albumId, int photoId)
```

Parameters

albumId [int](#)

Идентификатор альбома.

photoId [int](#)

Идентификатор фотографии.

Returns

[ResultVoid](#)

Результат выполнения операции (успех или ошибка).

Class ImportPhotoUseCase

Namespace: [PhotoCatalog.Application.UseCases](#)

Assembly: PhotoCatalog.Application.dll

Содержит сценарий оркестрации добавления нового фото в каталог.

```
public class ImportPhotoUseCase
```

Inheritance

[object](#)  ← ImportPhotoUseCase

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  ,
[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  ,
[object.ToString\(\)](#) 

Extension Methods

[ResultExtensions.ToResult<TValue>\(TValue\)](#) ,
[ResultExtensions.ToResult<TValue>\(TValue?, Error\)](#)

Remarks

UseCase включает работу с файловой системой (копирование, хэш) и базой данных. Обеспечивает атомарность операции через UnitOfWork и компенсационные механизмы.

Constructors

ImportPhotoUseCase(IFileStorage,
IFileMetadataExtractor, IPhotoRepository, IUnitOfWork,
ILogger<ImportPhotoUseCase>)

Инициализирует новый экземпляр класса [ImportPhotoUseCase](#).

```
public ImportPhotoUseCase(IFileStorage fileStorage, IFileMetadataExtractor  
metadataExtractor, IPhotoRepository photoRepository, IUnitOfWork unitOfWork,  
ILogger<ImportPhotoUseCase> logger)
```

Parameters

fileStorage [IFileStorage](#)

Сервис для работы с файловой системой.

metadataExtractor [IFileMetadataExtractor](#)

Сервис для извлечения метаданных файла.

photoRepository [IPhotoRepository](#)

Репозиторий для работы с сущностями Photo.

unitOfWork [IUnitOfWork](#)

Контракт для управления транзакциями.

logger [ILogger](#) <[ImportPhotoUseCase](#)>

Сервис для логирования.

Methods

Execute(ImportPhotoRequest)

Выполняет импорт фотографии в каталог.

```
public Result<PhotoResponse> Execute(ImportPhotoRequest request)
```

Parameters

request [ImportPhotoRequest](#)

Запрос на импорт фотографии.

Returns

[Result](#)<[PhotoResponse](#)>

Результат операции:

- Успех с [PhotoResponse](#).

- Ошибка с детализацией причины.

Class MoveFolderUseCase

Namespace: [PhotoCatalog.Application.UseCases](#)

Assembly: PhotoCatalog.Application.dll

Сценарий использования для перемещения папки.

```
public class MoveFolderUseCase
```

Inheritance

[object](#)  ← MoveFolderUseCase

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  ,
[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  ,
[object.ToString\(\)](#) 

Extension Methods

[ResultExtensions.ToResult<TValue>\(TValue\)](#) ,
[ResultExtensions.ToResult<TValue>\(TValue?, Error\)](#)

Constructors

MoveFolderUseCase(IFolderRepository,
IFolderHierarchyValidator, IUnitOfWork, ILogger)

Сценарий использования для перемещения папки.

```
public MoveFolderUseCase(IFolderRepository folderRepository,  
IFolderHierarchyValidator folderHierarchyValidator, IUnitOfWork unitOfWork,  
ILogger logger)
```

Parameters

folderRepository [IFolderRepository](#)

Репозиторий папок.

folderHierarchyValidator [IFolderHierarchyValidator](#)

Валидатор иерархии папок.

`unitOfWork` [IUnitOfWork](#)

Единица работы.

`logger` [ILogger](#)

Логгер.

Methods

Execute(int, int)

Выполняет перемещение папки с идентификатором `folderId` в родительскую папку с идентификатором `newParentId`.

```
public ResultVoid Execute(int folderId, int newParentId)
```

Parameters

`folderId` [int](#)

Идентификатор исходной папки.

`newParentId` [int](#)

Идентификатор целевой папки.

Returns

[ResultVoid](#)

Результат выполнения операции (успех или ошибка).

Namespace PhotoCatalog.Domain.Entities

Classes

[Album](#)

Представляет доменную сущность виртуального альбома.

[Folder](#)

Представляет папку.

[Photo](#)

Представляет основную сущность фотографии в системе, привязанную к физическому файлу.

[Tag](#)

Представляет тег для фотографий.

Class Album

Namespace: [PhotoCatalog.Domain.Entities](#)

Assembly: PhotoCatalog.Domain.dll

Представляет доменную сущность виртуального альбома.

```
public class Album
```

Inheritance

[object](#)  ← Album

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  ,
[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  ,
[object.ToString\(\)](#) 

Extension Methods

[ResultExtensions.ToResult<TValue>\(TValue\)](#) ,
[ResultExtensions.ToResult<TValue>\(TValue?, Error\)](#)

Remarks

Альбом отвечает за строгий контроль своего содержимого. Альбом проверяет, чтобы одну и ту же фотографию нельзя было прикрепить дважды.

Properties

FolderId

Получает идентификатор папки, в которой расположен альбом.

```
public int? FolderId { get; }
```

Property Value

[int](#) ?

Идентификатор папки или null, если альбом не перемещен в папку.

Id

Получает уникальный идентификатор альбома.

```
public int Id { get; }
```

Property Value

[int](#)

Name

Получает наименование альбома.

```
public string Name { get; }
```

Property Value

[string](#)

PhotoIds

Получает коллекцию идентификаторов фотографий, принадлежащих альбому, доступную только для чтения.

```
public IReadOnlyCollection<int> PhotoIds { get; }
```

Property Value

[IReadOnlyCollection](#) <[int](#)>

Methods

AddPhoto(int)

Добавляет фотографию в альбом.

```
public ResultVoid AddPhoto(int photoId)
```

Parameters

photoId [int](#)

Идентификатор добавляемой фотографии.

Returns

[ResultVoid](#)

Результат операции:

- Успех при успешном добавлении фотографии;
- Ошибка [DuplicatePhoto](#), если фотография уже есть в альбоме.

Create(string)

Создает новый экземпляр альбома с проверкой валидности наименования.

```
public static Result<Album> Create(string name)
```

Parameters

name [string](#)

Наименование создаваемого альбома.

Returns

[Result<Album>](#)

Результат операции:

- Успех с созданным альбомом;
- Ошибка [EmptyName](#), если наименование пустое.

MoveToFolder(Folder)

Перемещает альбом в указанную папку.

```
public ResultVoid MoveToFolder(Folder folder)
```

Parameters

folder [Folder](#)

Папка назначения.

Returns

[ResultVoid](#)

Успешный результат выполнения операции.

RemovePhoto(int)

Удаляет фотографию из альбома.

```
public ResultVoid RemovePhoto(int photoId)
```

Parameters

photoId [int](#)

Идентификатор удаляемой фотографии.

Returns

[ResultVoid](#)

Успешный результат выполнения операции.

Rename(string)

Изменяет наименование альбома на новое значение.

```
public ResultVoid Rename(string newName)
```

Parameters

`newName` [string](#) 

Новое наименование альбома.

Returns

[ResultVoid](#)

Результат операции:

- Успех при успешном переименовании;
- Ошибка [EmptyName](#), если новое наименование пустое.

Class Folder

Namespace: [PhotoCatalog.Domain.Entities](#)

Assembly: PhotoCatalog.Domain.dll

Представляет папку.

```
public class Folder
```

Inheritance

[object](#)  ← Folder

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  ,
[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  ,
[object.ToString\(\)](#) 

Extension Methods

[ResultExtensions.ToResult<TValue>\(TValue\)](#) ,
[ResultExtensions.ToResult<TValue>\(TValue?, Error\)](#)

Remarks

ВНИМАНИЕ: Не создавайте объект через конструктор по умолчанию.

Properties

Id

Идентификатор.

```
public int Id { get; }
```

Property Value

[int](#) 

Name

Название.

```
public string Name { get; }
```

Property Value

[string](#)

ParentFolderId

Идентификатор родителя.

```
public int? ParentFolderId { get; }
```

Property Value

[int](#)?

Methods

Create(int, string)

Фабричный метод по созданию экземпляров класса Folder.

```
public static Result<Folder> Create(int id, string name)
```

Parameters

id [int](#)

идентификатор.

name [string](#)

название.

Returns

[Result](#)<[Folder](#)>

Возвращает экземпляр.

MoveTo(Folder)

Перемещает папку к родительской папке.

```
public ResultVoid MoveTo(Folder parentFolder)
```

Parameters

parentFolder [Folder](#)

родительская папка.

Returns

[ResultVoid](#)

Возвращает результата выполнения.

MoveToRoot()

Перемещает папку в корень.

```
public ResultVoid MoveToRoot()
```

Returns

[ResultVoid](#)

Возвращает результата выполнения.

Rename(string)

Переименовать экземпляр.

```
public ResultVoid Rename(string newName)
```

Parameters

newName [string](#) 

новое название.

Returns

[ResultVoid](#)

Возвращает результат выполнения.

Class Photo

Namespace: [PhotoCatalog.Domain.Entities](#)

Assembly: PhotoCatalog.Domain.dll

Представляет основную сущность фотографии в системе, привязанную к физическому файлу.

```
public class Photo
```

Inheritance

[object](#)  ← Photo

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Extension Methods

[ResultExtensions.ToResult<TValue>\(TValue\)](#) ,
[ResultExtensions.ToResult<TValue>\(TValue?, Error\)](#)

Properties

AddedAt

Дата и время добавления фотографии в каталог (UTC).

```
public DateTime AddedAt { get; }
```

Property Value

[DateTime](#) 

Dimensions

Размеры изображения (ширина/высота).

```
public Dimensions Dimensions { get; }
```

Property Value

[Dimensions](#)

FileHash

контрольная-сумма файла для проверки целостности.

```
public string FileHash { get; }
```

Property Value

[string](#)

Id

Уникальный идентификатор фотографии.

```
public int Id { get; }
```

Property Value

[int](#)

RealPath

Полный путь к файлу на диске.

```
public string RealPath { get; }
```

Property Value

[string](#)

TagIds

Список идентификаторов тегов, присвоенных данной фотографии.

```
public IReadOnlyCollection<int> TagIds { get; }
```

Property Value

[IReadOnlyCollection](#) [<int>](#)

Methods

AddTag(int)

Добавляет тег к фотографии.

```
public ResultVoid AddTag(int tagId)
```

Parameters

tagId [int](#)

Идентификатор тега.

Returns

[ResultVoid](#)

Успех или ошибка [Photo.DuplicateTag](#), если тег уже добавлен.

Create(string)

Фабричный метод для создания нового экземпляра фотографии.

```
public static Result<Photo> Create(string realPath)
```

Parameters

`realPath` [string](#)

Путь к файлу на диске.

Returns

[Result](#)<[Photo](#)>

Результат выполнения операции с объектом [Photo](#) или ошибкой `Photo.EmptyPath`.

RemoveTag(int)

Удаляет тег из коллекции фотографии.

```
public ResultVoid RemoveTag(int tagId)
```

Parameters

`tagId` [int](#)

Идентификатор тега для удаления.

Returns

[ResultVoid](#)

Успех или ошибка `Photo.TagNotExists`, если тег не найден.

SetDimensions(Dimensions)

Устанавливает размеры изображения.

```
public ResultVoid SetDimensions(Dimensions newDimensions)
```

Parameters

`newDimensions` [Dimensions](#)

Размеры изображения.

Returns

[ResultVoid](#)

UpdateHash(string)

Обновляет хеш-сумму файла.

```
public ResultVoid UpdateHash(string newHash)
```

Parameters

newHash [string](#) 

Новое строковое значение хеша.

Returns

[ResultVoid](#)

Результат выполнения операции.

Class Tag

Namespace: [PhotoCatalog.Domain.Entities](#)

Assembly: PhotoCatalog.Domain.dll

Представляет тег для фотографий.

```
public class Tag
```

Inheritance

[object](#)  ← Tag

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  ,
[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  ,
[object.ToString\(\)](#) 

Extension Methods

[ResultExtensions.ToResult<TValue>\(TValue\)](#) ,
[ResultExtensions.ToResult<TValue>\(TValue?, Error\)](#)

Properties

Id

Уникальный идентификатор тега.

```
public int Id { get; }
```

Property Value

[int](#) 

Name

Нормализованное имя тега (в нижнем регистре).

```
public string Name { get; }
```

Property Value

[string](#)

Methods

Create(string)

Создает новый валидный тег.

```
public static Result<Tag> Create(string name)
```

Parameters

name [string](#)

Имя тега (будет нормализовано).

Returns

[Result<Tag>](#)

Результат создания с тегом или ошибкой.

Namespace PhotoCatalog.Domain.

Extensions

Classes

[ResultExtensions](#)

Методы расширения для последовательной обработки результатов, содержащих данные.

[ResultVoidExtensions](#)

Методы расширения для последовательной обработки результатов без данных.

Class ResultExtensions

Namespace: [PhotoCatalog.Domain.Extensions](#)

Assembly: PhotoCatalog.Domain.dll

Методы расширения для последовательной обработки результатов, содержащих данные.

```
public static class ResultExtensions
```

Inheritance

[object](#)  ← ResultExtensions

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Methods

Check<TValue>(Result<TValue>?, Func<TValue, ResultVoid>)

Выполняет проверку через стороннюю операцию без возврата данных. Сохраняет исходные данные для дальнейшей обработки в цепочке.

```
public static Result<TValue> Check<TValue>(this Result<TValue>? result, Func<TValue, ResultVoid> checker)
```

Parameters

result [Result](#)<TValue>

Текущий результат.

checker [Func](#)  <TValue, [ResultVoid](#)>

Функция проверки, возвращающая статус.

Returns

[Result](#)<TValue>

Исходный результат при успехе, либо провальный с ошибкой проверки.

Type Parameters

TValue

Тип сохраняемых данных.

Examples

```
Result<Document> result = documentResult
    .Check(doc => Security.ValidateSignature(doc))
    .Then(doc => Process(doc));
```

Check<TValue, TOther>(Result<TValue>?, Func<TValue, Result<TOther>>)

Выполняет проверку через операцию с возвратом данных. Игнорирует возвращаемые данные проверки, сохраняя только исходный объект в цепочке.

```
public static Result<TValue> Check<TValue, TOther>(this Result<TValue>? result,
    Func<TValue, Result<TOther>> checker)
```

Parameters

result [Result](#)<TValue>

Текущий результат.

checker [Func](#)<TValue, [Result](#)<TOther>>

Функция проверки, возвращающая сторонний результат.

Returns

[Result](#)<TValue>

Исходный результат при успехе, либо провальный с ошибкой проверки.

Type Parameters

TValue

Тип сохраняемых данных.

TOther

Тип игнорируемых данных проверки.

Examples

```
Result<Order> result = orderResult
    .Check(order => ValidateCustomer(order.CustomerId)) //
    Возвращает Result<Customer>
    .Then(order => Ship(order));
```

Ensure<TValue>(Result<TValue>?, Func<TValue, bool>, Error)

Проверяет успешный результат на соответствие заданному бизнес-правилу. Прерывает цепочку указанной ошибкой, если условие не выполнено.

```
public static Result<TValue> Ensure<TValue>(this Result<TValue>? result,
    Func<TValue, bool> predicate, Error error)
```

Parameters

result [Result](#)<TValue>

Текущий результат.

predicate [Func](#)<TValue, [bool](#)>

Функция-условие.

error [Error](#)

Ошибка, возвращаемая при несоблюдении условия.

Returns

[Result](#)<TValue>

Исходные данные при успехе, либо провальный результат с ошибкой.

Type Parameters

TValue

Тип проверяемых данных.

Examples

```
Result<int> validAge = ageResult
    .Ensure(
        age => age >= 18,
        new Error("Validation.Age", "Требуется совершеннолетие.")
    );
```

Finally<TValue, TLanding>(Result<TValue>?, Func<TValue, TLanding>, Func<Error, TLanding>)

Завершает цепочку операций, принудительно извлекая данные и конвертируя их в финальный тип.

```
public static TLanding Finally<TValue, TLanding>(this Result<TValue>? result,
    Func<TValue, TLanding> success, Func<Error, TLanding> failure)
```

Parameters

result [Result](#)<TValue>

Финальный результат.

success [Func](#)<TValue, TLanding>

Маппер для успешных данных.

failure [Func](#)<[Error](#), TLanding>

Маппер для ошибки.

Returns

TLanding

Объект целевого типа, созданный на основе успеха или провала.

Type Parameters

TValue

Тип данных в результате.

TLanding

Целевой тип возвращаемого значения.

Examples

```
IActionResult response = result.Finally(  
    success: entity => Ok(entity),  
    failure: error => BadRequest(error)  
);
```

OnFailure<TValue>(Result<TValue>?, Action<Error>)

Выполняет побочное действие только при наличии ошибки.

```
public static Result<TValue> OnFailure<TValue>(this Result<TValue>? result,  
Action<Error> action)
```

Parameters

result [Result](#)<TValue>

Текущий результат.

action [Action](#) [↗](#) <[Error](#)>

Действие для логирования или обработки ошибки.

Returns

[Result](#)<TValue>

Исходный результат без изменений.

Type Parameters

TValue

Тип данных.

Examples

```
result.OnFailure(error => Log.Error($"Сбой бизнес-логики: {error.Message}"));
```

OnSuccess<TValue>(Result<TValue>?, Action<TValue>)

Выполняет побочное действие (например, логирование) только при успешном результате.

```
public static Result<TValue> OnSuccess<TValue>(this Result<TValue>? result,  
Action<TValue> action)
```

Parameters

result [Result](#)<TValue>

Текущий результат.

action [Action](#)<TValue>

Действие, использующее успешные данные.

Returns

[Result](#)<TValue>

Исходный результат без изменений.

Type Parameters

TValue

Тип данных.

Examples

```
result.OnSuccess(entity => Log.Info($"Сущность {entity.Id} обработана"));
```

ThenTry<TValue>(Result<TValue>?, Action<TValue>, Func<Exception, Error>)

Безопасно выполняет рискованное действие с данными без возврата новых значений.

```
public static ResultVoid ThenTry<TValue>(this Result<TValue>? result, Action<TValue> nextStep, Func<Exception, Error> errorHandler)
```

Parameters

result [Result](#)<TValue>

Текущий результат.

nextStep [Action](#)<TValue>

Рискованное действие.

errorHandler [Func](#)<[Exception](#), [Error](#)>

Функция перехвата исключения.

Returns

[ResultVoid](#)

Успешный статус выполнения, либо провальный с перехваченной ошибкой.

Type Parameters

TValue

Тип текущих данных.

Examples

```
ResultVoid pushResult = entityResult
    .ThenTry(
        entity => MessageBus.Publish(entity),
        ex => new Error("Bus.PublishError", "Сбой брокера.")
    );
```

ThenTry<TValue, TNextValue>(Result<TValue>?, Func<TValue, TNextValue>, Func<Exception, Error>)

Безопасно выполняет следующий шаг вычислений, изолируя выброс исключений.

```
public static Result<TNextValue> ThenTry<TValue, TNextValue>(this Result<TValue>?
result, Func<TValue, TNextValue> nextStep, Func<Exception, Error> errorHandler)
```

Parameters

result [Result](#)<TValue>

Текущий результат.

nextStep [Func](#)<TValue, TNextValue>

Рискованная функция трансформации данных.

errorHandler [Func](#)<[Exception](#), [Error](#)>

Функция перехвата исключения.

Returns

[Result](#)<TNextValue>

Успешный результат с новыми данными, либо провальный с перехваченной ошибкой.

Type Parameters

TValue

Тип текущих данных.

TNextValue

Тип возвращаемых данных из рискованного кода.

Examples

```
Result<Data> parsedResult = jsonStringResult
    .ThenTry(
        json => JsonSerializer.Deserialize<Data>(json),
        ex => new Error("Json.ParseError", "Неверный формат.")
    );
```

Then<TValue>(Result<TValue>?, Func<TValue, ResultVoid>)

Выполняет переход от результата с данными к операции, не возвращающей значения.

```
public static ResultVoid Then<TValue>(this Result<TValue>? result, Func<TValue,
ResultVoid> nextStep)
```

Parameters

result [Result](#)<TValue>

Текущий результат.

nextStep [Func](#)<TValue, [ResultVoid](#)>

Функция, возвращающая результат без данных.

Returns

[ResultVoid](#)

Статус выполнения следующего шага, либо проброшенная ошибка.

Type Parameters

TValue

Тип текущих данных.

Examples

```
ResultVoid saveResult = CreateEntity()  
    .Then(entity => Repository.Save(entity));
```

Then<TValue, TNextValue>(Result<TValue>?, Func<TValue, Result<TNextValue>>)

Выполняет переход от текущих данных к новым данным. Следующий шаг выполняется только при успешном статусе текущего результата.

```
public static Result<TNextValue> Then<TValue, TNextValue>(this Result<TValue>?  
result, Func<TValue, Result<TNextValue>> nextStep)
```

Parameters

result [Result](#)<TValue>

Текущий результат.

nextStep [Func](#)<TValue, [Result](#)<TNextValue>>

Функция, возвращающая новый результат вычислений.

Returns

[Result](#)<TNextValue>

Результат выполнения следующего шага, либо проброшенная ошибка.

Type Parameters

TValue

Исходный тип данных.

TNextValue

Тип данных следующего шага.

Examples

```
Result<Details> result = GetEntity()  
    .Then(entity => FetchDetails(entity.Id));
```

ToResult<TValue>(TValue)

Создает успешный результат из существующего значения.

```
public static Result<TValue> ToResult<TValue>(this TValue value)
```

Parameters

value TValue

Значение для инициализации цепочки.

Returns

[Result](#)<TValue>

Успешный результат с данными, либо провальный с системной ошибкой пустого значения.

Type Parameters

TValue

Тип преобразуемого значения.

Examples

```
Result<string> result = "initial context".ToResult();
```

ToResult<TValue>(TValue?, Error)

Безопасно преобразует потенциально пустое значение в результат, заменяя null на ошибку.

```
public static Result<TValue> ToResult<TValue>(this TValue? value, Error errorIfNull)
```

Parameters

value TValue

Значение, полученное из внешнего источника.

errorIfNull [Error](#)

Доменная ошибка, сигнализирующая об отсутствии данных.

Returns

[Result](#)<TValue>

Успешный результат с данными, либо провальный с переданной ошибкой.

Type Parameters

TValue

Тип преобразуемого значения.

Examples

```
Result<Entity> result = repository.Find(id)  
    .ToResult(new Error("Entity.NotFound", "Запись не найдена"));
```

Transform<TValue, TNextValue>(Result<TValue>?, Func<TValue, TNextValue>)

Безопасно преобразует данные в новый формат. Функция маппинга не генерирует ошибок домена.

```
public static Result<TNextValue> Transform<TValue, TNextValue>(this Result<TValue>?  
result, Func<TValue, TNextValue> mapper)
```

Parameters

result [Result](#)<TValue>

Текущий результат.

mapper [Func](#)<TValue, TNextValue>

Функция трансформации.

Returns

[Result](#)<TNextValue>

Результат с преобразованными данными, либо проброшенная ошибка.

Type Parameters

TValue

Исходный тип данных.

TNextValue

Целевой тип данных.

Examples

```
Result<Guid> idResult = entityResult  
    .Transform(entity => entity.Id);
```

TryCatch<TValue>(Func<TValue>, Func<Exception, Error>)

Иницирует безопасное выполнение функции, перехватывая любые исключения внешнего кода. Используется как стартовая фабрика, а не метод расширения.

```
public static Result<TValue> TryCatch<TValue>(Func<TValue> action, Func<Exception, Error> errorHandler)
```

Parameters

action [Func](#)<TValue>

Рискованная функция получения данных.

errorHandler [Func](#)<[Exception](#), [Error](#)>

Функция, преобразующая исключение в доменную ошибку.

Returns

[Result](#)<TValue>

Успешный результат с данными, либо провальный с перехваченной ошибкой.

Type Parameters

TValue

Тип возвращаемых данных.

Examples

```
Result<string> configResult = ResultExtensions.TryCatch(  
    () => File.ReadAllText("config.json"),  
    ex => new Error("Config.ReadFailed", ex.Message)  
);
```

Class ResultVoidExtensions

Namespace: [PhotoCatalog.Domain.Extensions](#)

Assembly: PhotoCatalog.Domain.dll

Методы расширения для последовательной обработки результатов без данных.

```
public static class ResultVoidExtensions
```

Inheritance

[object](#)  ← ResultVoidExtensions

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  ,
[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  ,
[object.ToString\(\)](#) 

Methods

Finally<TLanding>(ResultVoid, Func<TLanding>, Func<Error, TLanding>)

Завершает цепочку операций, принудительно конвертируя результат в финальный тип данных.

```
public static TLanding Finally<TLanding>(this ResultVoid result, Func<TLanding>  
    success, Func<Error, TLanding> failure)
```

Parameters

result [ResultVoid](#)

Финальный объект результата.

success [Func](#)  <TLanding>

Функция, возвращающая значение при успехе.

failure [Func](#)  <[Error](#), TLanding>

Функция, преобразующая ошибку в финальное значение.

Returns

TLanding

Единое значение указанного типа, независимое от статуса успеха.

Type Parameters

TLanding

Тип возвращаемого значения (например, HTTP-статус).

Examples

```
int statusCode = result.Finally(  
    success: () => 204, // No Content  
    failure: error => 400 // Bad Request  
);
```

OnFailure(ResultVoid, Action<Error>)

Выполняет побочное действие исключительно при наличии ошибки в результате.

```
public static ResultVoid OnFailure(this ResultVoid result, Action<Error> action)
```

Parameters

result [ResultVoid](#)

Текущий результат.

action [Action](#) [Error](#)

Действие для обработки ошибки.

Returns

[ResultVoid](#)

Исходный объект результата без изменений.

Examples

```
result.OnFailure(error => Logger.Error($"Сбой: {error.Code}"));
```

OnSuccess(ResultVoid, Action)

Выполняет побочное действие исключительно при успешном статусе результата.

```
public static ResultVoid OnSuccess(this ResultVoid result, Action action)
```

Parameters

result [ResultVoid](#)

Текущий результат.

action [Action](#) 

Действие для выполнения.

Returns

[ResultVoid](#)

Исходный объект результата без изменений.

Examples

```
result.OnSuccess(() => Logger.Log("Операция успешно завершена."));
```

Then(ResultVoid, Func<ResultVoid>)

Связывает две операции, не возвращающие данных. Выполняет следующий шаг только в случае успеха текущего результата.

```
public static ResultVoid Then(this ResultVoid result, Func<ResultVoid> nextStep)
```

Parameters

result [ResultVoid](#)

Текущий результат без значения.

nextStep [Func](#) [<ResultVoid>](#)

Функция, возвращающая следующий результат выполнения.

Returns

[ResultVoid](#)

Результат выполнения следующего шага или провальный результат с текущей ошибкой.

Examples

```
ResultVoid result = ValidationService.CheckPermissions()  
    .Then(() => Database.CommitTransaction());
```

ThenTry(ResultVoid, Action, Func<Exception, Error>)

Безопасно выполняет следующее рискованное действие без возврата данных.

```
public static ResultVoid ThenTry(this ResultVoid result, Action nextStep,  
    Func<Exception, Error> errorHandler)
```

Parameters

result [ResultVoid](#)

Текущий результат.

nextStep [Action](#)

Рискованное действие (например, вызов стороннего API).

errorHandler [Func](#) [<Exception, Error>](#)

Функция перехвата и преобразования исключения.

Returns

[ResultVoid](#)

Успешный результат, либо провальный с ошибкой (текущей или из перехватчика).

Examples

```
ResultVoid notifyResult = commitResult
    .ThenTry(
        () => externalService.SendNotification(),
        ex => new Error("Network.SendFailed", "Сбой отправки уведомления.")
    );
```

ThenTry<TNextValue>(ResultVoid, Func<TNextValue>, Func<Exception, Error>)

Безопасно выполняет следующий шаг с получением данных, изолируя возможные исключения.

```
public static Result<TNextValue> ThenTry<TNextValue>(this ResultVoid result,
    Func<TNextValue> nextStep, Func<Exception, Error> errorHandler)
```

Parameters

result [ResultVoid](#)

Текущий результат.

nextStep [Func](#) <TNextValue>

Рискованная функция вычисления данных.

errorHandler [Func](#) <[Exception](#), [Error](#)>

Функция перехвата и преобразования исключения.

Returns

[Result](#)<TNextValue>

Успешный результат с данными, либо провальный с ошибкой (текущей или из перехватчика).

Type Parameters

TNextValue

Ожидаемый тип возвращаемых данных.

Examples

```
Result<byte[]> fileResult = validationResult
    .ThenTry(
        () => File.ReadAllBytes(path),
        ex => new Error("File.ReadError", "Сбой чтения файла.")
    );
```

Then<TNextValue>(ResultVoid, Func<Result<TNextValue>>)

Осуществляет переход от операции без результата к операции с возвращаемыми данными. Выполняет вычисление только при успешном завершении предыдущего шага.

```
public static Result<TNextValue> Then<TNextValue>(this ResultVoid result,
    Func<Result<TNextValue>> nextStep)
```

Parameters

result [ResultVoid](#)

Текущий результат без значения.

nextStep [Func](#) [Result](#)<TNextValue>>

Функция, возвращающая результат с данными.

Returns

[Result](#)<TNextValue>

Результат выполнения следующего шага или провальный результат с текущей ошибкой.

Type Parameters

TNextValue

Тип значения, которое вернёт следующий шаг.

Examples

```
Result<Entity> result = Security.ValidateAccess()  
    .Then(() => Repository.GetEntity(id));
```

TryCatch(Action, Func<Exception, Error>)

Безопасно инициирует цепочку вычислений, перехватывая любые исключения внешнего кода. Используется как стартовая фабрика, а не метод расширения.

```
public static ResultVoid TryCatch(Action action, Func<Exception,  
Error> errorHandler)
```

Parameters

action [Action](#)

Рискованное действие без возвращаемого значения.

errorHandler [Func](#) <[Exception](#), [Error](#)>

Функция, преобразующая пойманное исключение в доменную ошибку.

Returns

[ResultVoid](#)

Успешный результат при отсутствии исключений, либо провальный с ошибкой из обработчика.

Examples

```
ResultVoid saveResult = ResultVoidExtensions.TryCatch(  
    () => _dbContext.SaveChanges(),  
    ex => new Error("Database.SaveError", ex.Message)  
);
```

Namespace PhotoCatalog.Domain. Interfaces.Repositories

Interfaces

[IAlbumRepository](#)

Предоставляет методы для управления виртуальными альбомами в репозитории.

[IFolderRepository](#)

Интерфейс репозитория для управления иерархией виртуальных папок.

[IPhotoRepository](#)

Интерфейс репозитория для работы с сущностью Photo. Определяет операции для получения и сохранения фотографий. Использует паттерн Result для безопасной передачи инфраструктурных ошибок.

[ITagRepository](#)

Репозиторий для работы с тегами.

Interface IAlbumRepository

Namespace: [PhotoCatalog.Domain.Interfaces.Repositories](#)

Assembly: PhotoCatalog.Domain.dll

Предоставляет методы для управления виртуальными альбомами в репозитории.

```
public interface IAlbumRepository
```

Extension Methods

[ResultExtensions.ToResult<TValue>\(TValue\)](#) ,
[ResultExtensions.ToResult<TValue>\(TValue?, Error\)](#)

Remarks

Этот интерфейс определяет контракт для операций CRUD (создание, чтение, обновление, удаление) с объектами типа [Album](#). Все методы возвращают результат, обернутый в Result или [Result<T>](#), что позволяет единообразно обрабатывать успешные и ошибочные сценарии.

Methods

Add(Album)

Добавляет новый альбом в репозиторий.

```
ResultVoid Add(Album album)
```

Parameters

album [Album](#)

Объект альбома для добавления. Не может быть null.

Returns

[ResultVoid](#)

Объект [ResultVoid](#), указывающий на успешность выполнения операции. Возвращает успешный результат, если альбом успешно добавлен, или неуспешный с описанием ошибки в противном случае.

Remarks

Перед добавлением может выполняться проверка уникальности имени альбома или других бизнес-правил. В случае нарушения этих правил возвращается ошибка.

Delete(int)

Удаляет альбом из репозитория по его идентификатору.

```
ResultVoid Delete(int id)
```

Parameters

id [int](#)

Уникальный идентификатор альбома для удаления.

Returns

[ResultVoid](#)

Объект [ResultVoid](#), указывающий на успешность выполнения операции. Возвращает успешный результат, если альбом успешно удален, или неуспешный с описанием ошибки (например, если альбом не найден).

Remarks

При удалении альбома может потребоваться также удалить или обновить связанные данные (например, фотографии или ссылки на этот альбом). Рекомендуется реализовать каскадное удаление или соответствующую бизнес-логику.

GetById(int)

Получает альбом по его уникальному идентификатору.

```
Result<Album> GetById(int id)
```

Parameters

id [int](#)

Уникальный идентификатор альбома.

Returns

[Result<Album>](#)

Объект [Result<T>](#), содержащий найденный альбом в случае успеха, или информацию об ошибке, если альбом не найден или произошла ошибка доступа к данным.

Remarks

Если альбом с указанным идентификатором не существует, возвращается неуспешный Result с соответствующим сообщением об ошибке.

Update(Album)

Обновляет существующий альбом в репозитории.

```
ResultVoid Update(Album album)
```

Parameters

album [Album](#)

Объект альбома с обновленными данными. Не может быть null.

Returns

[ResultVoid](#)

Объект [ResultVoid](#), указывающий на успешность выполнения операции. Возвращает успешный результат, если альбом успешно обновлен, или неуспешный с описанием ошибки (например, если альбом не найден).

Remarks

Обновление выполняется только для существующего альбома. Если альбом с указанным идентификатором не найден в репозитории, возвращается ошибка.

Interface IFolderRepository

Namespace: [PhotoCatalog.Domain.Interfaces.Repositories](#)

Assembly: PhotoCatalog.Domain.dll

Интерфейс репозитория для управления иерархией виртуальных папок.

```
public interface IFolderRepository
```

Extension Methods

[ResultExtensions.ToResult<TValue>\(TValue\)](#) ,
[ResultExtensions.ToResult<TValue>\(TValue?, Error\)](#)

Methods

Add(Folder)

Добавляет новую папку в репозиторий.

```
ResultVoid Add(Folder folder)
```

Parameters

folder [Folder](#)

Папка для добавления.

Returns

[ResultVoid](#)

Результат операции:

- Успех, если папка успешно добавлена
- Инфраструктурную ошибку при сбое базы данных

Delete(int)

Удаляет папку по идентификатору.

```
ResultVoid Delete(int id)
```

Parameters

id [int](#)

Идентификатор папки для удаления.

Returns

[ResultVoid](#)

Результат операции:

- Успех, если папка успешно удалена
- Ошибка NotFound, если папка не найдена
- Инфраструктурную ошибку при сбое базы данных

GetById(int)

Получить папку по идентификатору.

```
Result<Folder> GetById(int id)
```

Parameters

id [int](#)

Идентификатор папки.

Returns

[Result](#)<[Folder](#)>

Результат операции:

- Успех с папкой, если она найдена
- Ошибка NotFound, если папка не найдена
- Инфраструктурную ошибку при сбое базы данных

Update(Folder)

Обновляет существующую папку.

```
ResultVoid Update(Folder folder)
```

Parameters

folder [Folder](#)

Обновленная папка.

Returns

[ResultVoid](#)

Результат операции:

- Успех, если папка успешно обновлена
- Ошибка NotFound, если папка не найдена
- Инфраструктурную ошибку при сбое базы данных

Interface IPhotoRepository

Namespace: [PhotoCatalog.Domain.Interfaces.Repositories](#)

Assembly: PhotoCatalog.Domain.dll

Интерфейс репозитория для работы с сущностью Photo. Определяет операции для получения и сохранения фотографий. Использует паттерн Result для безопасной передачи инфраструктурных ошибок.

```
public interface IPhotoRepository
```

Extension Methods

[ResultExtensions.ToResult<TValue>\(TValue\)](#) ,
[ResultExtensions.ToResult<TValue>\(TValue?, Error\)](#)

Methods

Add(Photo)

Добавляет новую фотографию в репозиторий.

```
ResultVoid Add(Photo photo)
```

Parameters

photo [Photo](#)

Фото для добавления.

Returns

[ResultVoid](#)

Результат операции:

- Успех, если фотография успешно добавлена
- Инфраструктурную ошибку при сбое базы данных

Delete(int)

Удаляет фотографию по идентификатору.

```
ResultVoid Delete(int id)
```

Parameters

id [int](#)

Идентификатор фото для удаления.

Returns

[ResultVoid](#)

Результат операции:

- Успех, если фотография успешно удалена
- Ошибка NotFound, если фотография не найдена
- Инфраструктурную ошибку при сбое базы данных

GetByAlbumId(int)

Получает все фотографии альбома.

```
Result<IReadOnlyCollection<Photo>> GetByAlbumId(int albumId)
```

Parameters

albumId [int](#)

Идентификатор альбома.

Returns

[Result](#)<[IReadOnlyCollection](#) <[Photo](#)>>

Результат операции:

- Успех с неизменяемой коллекцией фотографий альбома (может быть пустой)

- Инфраструктурную ошибку при сбое базы данных

GetById(int)

Получает фотографию по идентификатору.

```
Result<Photo> GetById(int id)
```

Parameters

id [int](#)

Идентификатор фотографии.

Returns

[Result](#)<[Photo](#)>

Результат операции:

- Успех с фотографией, если она найдена
- Ошибка NotFound, если фотография не найдена
- Инфраструктурную ошибку при сбое базы данных

GetByPath(string)

Получает фотографию по реальному пути к файлу.

```
Result<Photo> GetByPath(string realPath)
```

Parameters

realPath [string](#)

Реальный путь к файлу фотографии.

Returns

[Result](#)<[Photo](#)>

Результат операции:

- Успех с фотографией, если она найдена
- Ошибка NotFound, если фотография не найдена
- Инфраструктурную ошибку при сбое базы данных

GetByTags(IEnumerable<int>)

Получает фотографии по списку идентификаторов тегов.

```
Result<ICollection<Photo>> GetByTags(IEnumerable<int> tagIds)
```

Parameters

tagIds [IEnumerable](#) [<int>](#)

Идентификаторы тегов.

Returns

[Result](#) [<ICollection](#) [<Photo>>](#)

Результат операции:

- Успех с неизменяемой коллекцией фотографий, содержащих указанные теги (может быть пустой)
- Инфраструктурную ошибку при сбое базы данных

Update(Photo)

Обновляет существующую фотографию.

```
ResultVoid Update(Photo photo)
```

Parameters

photo [Photo](#)

Обновленное фото.

Returns

[ResultVoid](#)

Результат операции:

- Успех, если фотография успешно обновлена
- Ошибка NotFound, если фотография не найдена
- Инфраструктурную ошибку при сбое базы данных

Interface ITagRepository

Namespace: [PhotoCatalog.Domain.Interfaces.Repositories](#)

Assembly: PhotoCatalog.Domain.dll

Репозиторий для работы с тегами.

```
public interface ITagRepository
```

Extension Methods

[ResultExtensions.ToResult<TValue>\(TValue\)](#) ,
[ResultExtensions.ToResult<TValue>\(TValue?, Error\)](#)

Methods

Add(Tag)

Добавляет тег.

```
ResultVoid Add(Tag tag)
```

Parameters

tag [Tag](#)

Тег для добавления

Returns

[ResultVoid](#)

Результат операции

Delete(int)

Удаляет тег по ID.

```
Result<Tag> Delete(int id)
```

Parameters

id [int](#)

ID тега

Returns

[Result](#)<[Tag](#)>

Удаленный тег или ошибка

GetById(int)

Получает тег по ID.

```
Result<Tag> GetById(int id)
```

Parameters

id [int](#)

ID тега

Returns

[Result](#)<[Tag](#)>

Тег или ошибка

GetByName(string)

Получает тег по имени.

```
Result<Tag> GetByName(string name)
```

Parameters

name [string](#) 

Имя тега

Returns

[Result](#)<[Tag](#)>

Тег или ошибка

Namespace PhotoCatalog.Domain. Interfaces.Services

Interfaces

[IFileMetadataExtractor](#)

Контракт для извлечения метаданных из файлов.

[IFileStorage](#)

Хранилище файлов.

[IFolderHierarchyValidator](#)

Интерфейс валидатора дерева, который обеспечивает защиту от циклических зависимостей графа.

[IUnitOfWork](#)

Контракт для управления транзакциями в доменном слое.

Interface IFileMetadataExtractor

Namespace: [PhotoCatalog.Domain.Interfaces.Services](#)

Assembly: PhotoCatalog.Domain.dll

Контракт для извлечения метаданных из файлов.

```
public interface IFileMetadataExtractor
```

Extension Methods

[ResultExtensions.ToResult<TValue>\(TValue\)](#) ,
[ResultExtensions.ToResult<TValue>\(TValue?, Error\)](#)

Methods

CalculateHash(string)

Возвращает хэш файла по указанному пути.

```
Result<string> CalculateHash(string filePath)
```

Parameters

filePath [string](#) 

Путь к файлу.

Returns

[Result](#)<[string](#)  >

Хэш файла или ошибку.

GetDimensions(string)

Возвращает размеры файла (например, ширина и высота).

```
Result<Dimensions> GetDimensions(string filePath)
```

Parameters

filePath [string](#) 

Путь к файлу.

Returns

[Result](#)<[Dimensions](#)>

Размеры файла или ошибку.

Interface IFileStorage

Namespace: [PhotoCatalog.Domain.Interfaces.Services](#)

Assembly: PhotoCatalog.Domain.dll

Хранилище файлов.

```
public interface IFileStorage
```

Extension Methods

[ResultExtensions.ToResult<TValue>\(TValue\)](#) ,
[ResultExtensions.ToResult<TValue>\(TValue?, Error\)](#)

Methods

DeleteFile(string)

Удаляет файл по пути.

```
ResultVoid DeleteFile(string filePath)
```

Parameters

filePath [string](#) 

Путь к файлу для удаления

Returns

[ResultVoid](#)

Результат операции

FileExists(string)

Проверяет, существует ли файл по указанному пути.

```
Result<bool> FileExists(string filePath)
```

Parameters

filePath [string](#)

Путь к файлу

Returns

[Result<bool>](#)

True или false и ошибка при неудаче

StoreFile(string, string)

Сохраняет файл из исходного пути с новым именем.

```
Result<string> StoreFile(string sourcePath, string newFileName)
```

Parameters

sourcePath [string](#)

Путь к исходному файлу

newFileName [string](#)

Новое имя файла

Returns

[Result<string>](#)

Путь к сохранённому файлу или ошибка

Interface IFolderHierarchyValidator

Namespace: [PhotoCatalog.Domain.Interfaces.Services](#)

Assembly: PhotoCatalog.Domain.dll

Интерфейс валидатора дерева, который обеспечивает защиту от циклических зависимостей графа.

```
public interface IFolderHierarchyValidator
```

Extension Methods

[ResultExtensions.ToResult<TValue>\(TValue\)](#) ,
[ResultExtensions.ToResult<TValue>\(TValue?, Error\)](#)

Methods

CheckForCycles(int, int)

Метод проверки на циклические зависимости папки в дереве.

```
ResultVoid CheckForCycles(int sourceFolderId, int targetFolderId)
```

Parameters

sourceFolderId [int](#)

Идентификатор перемещаемой папки.

targetFolderId [int](#)

Идентификатор папки, в которую планируется перемещение.

Returns

[ResultVoid](#)

Успешный результат, если цикл не обнаружен; результат с ошибкой `ApplicationErrors.Folders.CycleDetected` в противном случае.

Interface IUnitOfWork

Namespace: [PhotoCatalog.Domain.Interfaces.Services](#)

Assembly: PhotoCatalog.Domain.dll

Контракт для управления транзакциями в доменном слое.

```
public interface IUnitOfWork : IDisposable
```

Inherited Members

[IDisposable.Dispose\(\)](#) 

Extension Methods

[ResultExtensions.ToResult<TValue>\(TValue\)](#) ,
[ResultExtensions.ToResult<TValue>\(TValue?, Error\)](#)

Remarks

Очищен от инфраструктурных типов для независимости слоя Domain.

Methods

BeginTransaction()

Начинает новую транзакцию.

```
ResultVoid BeginTransaction()
```

Returns

[ResultVoid](#)

Результат операции. Успех или ошибка с детализацией.

Commit()

Фиксирует все изменения текущей транзакции.

ResultVoid **Commit**()

Returns

[ResultVoid](#)

Результат операции. Успех или ошибка с детализацией.

Rollback()

Отменяет все изменения текущей транзакции.

ResultVoid **Rollback**()

Returns

[ResultVoid](#)

Результат операции. Успех или ошибка с детализацией.

Namespace PhotoCatalog.Domain.

Primitives

Classes

[DomainErrors](#)

Единый статический класс (реестр), который содержит все возможные бизнес-ошибки предметной области в виде заранее определенных структур [Error](#)>.

[DomainErrors.Album](#)

Ошибки для [DomainErrors.Album](#)

[DomainErrors.Dimensions](#)

Ошибки для [DomainErrors.Dimensions](#).

[DomainErrors.Folder](#)

Ошибки для [DomainErrors.Folder](#).

[DomainErrors.Photo](#)

Ошибки для [DomainErrors.Photo](#).

[DomainErrors.Tag](#)

Ошибки для [DomainErrors.Tag](#).

[Result<T>](#)

Представляет результат выполнения операции, возвращающей значение типа **T**.

[SystemErrors](#)

Реестр критических системных ошибок, которые не связаны с бизнес-правилами, а возникают из-за сбоев в потоке выполнения или инфраструктуре.

Structs

[Error](#)

Представляет структуру для стандартизированного описания бизнес-ошибок в домене.

[ResultVoid](#)

Представляет результат выполнения операции, не возвращающей значение (аналог void). Инкапсулирует логику успеха или провала.

Class DomainErrors

Namespace: [PhotoCatalog.Domain.Primitives](#)

Assembly: PhotoCatalog.Domain.dll

Единый статический класс (реестр), который содержит все возможные бизнес-ошибки предметной области в виде заранее определенных структур [Error](#)>.

```
public static class DomainErrors
```

Inheritance

[object](#) < DomainErrors

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) ,
[object.GetType\(\)](#) , [object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) ,
[object.ToString\(\)](#)

Class DomainErrors.Album

Namespace: [PhotoCatalog.Domain.Primitives](#)

Assembly: PhotoCatalog.Domain.dll

Ошибки для [DomainErrors.Album](#)

```
public static class DomainErrors.Album
```

Inheritance

[object](#)  ← DomainErrors.Album

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  ,
[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  ,
[object.ToString\(\)](#) 

Fields

DuplicatePhoto

Ошибка, когда эта фотография уже находится в данном альбоме.

```
public static readonly Error DuplicatePhoto
```

Field Value

[Error](#)

EmptyName

Ошибка, когда имя альбома пустое.

```
public static readonly Error EmptyName
```

Field Value

[Error](#)

Class DomainErrors.Dimensions

Namespace: [PhotoCatalog.Domain.Primitives](#)

Assembly: PhotoCatalog.Domain.dll

Ошибки для [DomainErrors.Dimensions](#).

```
public static class DomainErrors.Dimensions
```

Inheritance

[object](#)  ← DomainErrors.Dimensions

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  ,
[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  ,
[object.ToString\(\)](#) 

Fields

Invalid

Ошибка, когда ширина и высота меньше или равны нулю, или меньше разрешенного предела размера.

```
public static readonly Error Invalid
```

Field Value

[Error](#)

Class DomainErrors.Folder

Namespace: [PhotoCatalog.Domain.Primitives](#)

Assembly: PhotoCatalog.Domain.dll

Ошибки для [DomainErrors.Folder](#).

```
public static class DomainErrors.Folder
```

Inheritance

[object](#)  ← DomainErrors.Folder

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  ,
[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  ,
[object.ToString\(\)](#) 

Fields

CannotMoveToSelf

Ошибка, когда папка перемещается внутрь самой себя (циклическая ссылка).

```
public static readonly Error CannotMoveToSelf
```

Field Value

[Error](#)

EmptyName

Ошибка, когда имя папки пустое.

```
public static readonly Error EmptyName
```

Field Value

[Error](#)

Class DomainErrors.Photo

Namespace: [PhotoCatalog.Domain.Primitives](#)

Assembly: PhotoCatalog.Domain.dll

Ошибки для [DomainErrors.Photo](#).

```
public static class DomainErrors.Photo
```

Inheritance

[object](#)  ← DomainErrors.Photo

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  ,
[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  ,
[object.ToString\(\)](#) 

Fields

DuplicateTag

Ошибка, когда данный тег уже привязан к этой фотографии.

```
public static readonly Error DuplicateTag
```

Field Value

[Error](#)

EmptyPath

Ошибка, когда путь к файлу фотографии пустой.

```
public static readonly Error EmptyPath
```

Field Value

[Error](#)

TagNotExists

Ошибка, когда данный тег не существует в этой фотографии.

```
public static readonly Error TagNotExists
```

Field Value

[Error](#)

Class DomainErrors.Tag

Namespace: [PhotoCatalog.Domain.Primitives](#)

Assembly: PhotoCatalog.Domain.dll

Ошибки для [DomainErrors.Tag](#).

```
public static class DomainErrors.Tag
```

Inheritance

[object](#)  ← DomainErrors.Tag

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  ,
[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  ,
[object.ToString\(\)](#) 

Fields

EmptyName

Ошибка, когда имя тега пустое или состоит только из пробелов.

```
public static readonly Error EmptyName
```

Field Value

[Error](#)

TooLong

Ошибка, когда имя тега превышает 50 символов.

```
public static readonly Error TooLong
```

Field Value

[Error](#)

Struct Error

Namespace: [PhotoCatalog.Domain.Primitives](#)

Assembly: PhotoCatalog.Domain.dll

Представляет структуру для стандартизированного описания бизнес-ошибок в домене.

```
public readonly record struct Error : IEquatable<Error>
```

Implements

[IEquatable](#)[↗] [<Error>](#)

Inherited Members

[ValueType.Equals\(object\)](#)[↗] , [ValueType.GetHashCode\(\)](#)[↗] , [ValueType.ToString\(\)](#)[↗] ,
[object.Equals\(object, object\)](#)[↗] , [object.GetType\(\)](#)[↗] ,
[object.ReferenceEquals\(object, object\)](#)[↗]

Extension Methods

[ResultExtensions.ToResult<TValue>\(TValue\)](#) ,
[ResultExtensions.ToResult<TValue>\(TValue?, Error\)](#)

Constructors

Error(string, string)

Представляет структуру для стандартизированного описания бизнес-ошибок в домене.

```
public Error(string Code, string Message)
```

Parameters

Code [string](#)[↗]

Уникальный строковый идентификатор ошибки (например, "Tag.EmptyName").
Используется для программной проверки типа ошибки на прикладном слое.

Message [string](#)[↗]

Понятное описание проблемы на естественном языке. В первую очередь предназначено для логирования и помощи разработчикам при отладке.

Fields

None

Специальный объект, обозначающий отсутствие ошибки.

```
public static readonly Error None
```

Field Value

[Error](#)

Properties

Code

Уникальный строковый идентификатор ошибки (например, "Tag.EmptyName").
Используется для программной проверки типа ошибки на прикладном слое.

```
public string Code { get; init; }
```

Property Value

[string](#)

Message

Понятное описание проблемы на естественном языке. В первую очередь предназначено для логирования и помощи разработчикам при отладке.

```
public string Message { get; init; }
```

Property Value

Struct ResultVoid

Namespace: [PhotoCatalog.Domain.Primitives](#)

Assembly: PhotoCatalog.Domain.dll

Представляет результат выполнения операции, не возвращающей значение (аналог void). Инкапсулирует логику успеха или провала.

```
public readonly record struct ResultVoid : IEquatable<ResultVoid>
```

Implements

[IEquatable](#)[↗] <[ResultVoid](#)>

Inherited Members

[ValueType.Equals\(object\)](#)[↗] , [ValueType.GetHashCode\(\)](#)[↗] , [ValueType.ToString\(\)](#)[↗] ,
[object.Equals\(object, object\)](#)[↗] , [object.GetType\(\)](#)[↗] ,
[object.ReferenceEquals\(object, object\)](#)[↗]

Extension Methods

[ResultExtensions.ToResult<TValue>\(TValue\)](#) ,
[ResultExtensions.ToResult<TValue>\(TValue?, Error\)](#) ,
[ResultVoidExtensions.Finally<TLanding>\(ResultVoid, Func<TLanding>, Func<Error, TLanding>\)](#) ,
[ResultVoidExtensions.OnFailure\(ResultVoid, Action<Error>\)](#) ,
[ResultVoidExtensions.OnSuccess\(ResultVoid, Action\)](#) ,
[ResultVoidExtensions.Then\(ResultVoid, Func<ResultVoid>\)](#) ,
[ResultVoidExtensions.Try\(Action, Func<Exception, Error>\)](#) ,
[ResultVoidExtensions.Try<TNextValue>\(ResultVoid, Func<TNextValue>, Func<Exception, Error>\)](#) ,
[ResultVoidExtensions.Then<TNextValue>\(ResultVoid, Func<Result<TNextValue>>\)](#)

Remarks

ВНИМАНИЕ: Не создавайте объект через конструктор по умолчанию. Для инициализации используйте исключительно статические методы: [Success\(\)](#) или [Failure\(Error\)](#).

Properties

Error

Объект детализированной ошибки. Если операция успешна, содержит [None](#).

```
public Error Error { get; }
```

Property Value

[Error](#)

IsFailure

Указывает, завершилась ли операция с ошибкой.

```
public bool IsFailure { get; }
```

Property Value

[bool](#)

IsSuccess

Указывает, завершилась ли операция успешно.

```
public bool IsSuccess { get; }
```

Property Value

[bool](#)

Methods

Failure(Error)

Создает провальный результат с указанной ошибкой.

```
public static ResultVoid Failure(Error error)
```

Parameters

error [Error](#)

Бизнес-ошибка, объясняющая причину провала.

Returns

[ResultVoid](#)

Провальный [ResultVoid](#), где [IsFailure](#) равно true, а [Error](#) содержит переданную ошибку.

Success()

Создает успешный результат без ошибок.

```
public static ResultVoid Success()
```

Returns

[ResultVoid](#)

Успешный [ResultVoid](#), где [IsSuccess](#) равно true, а [Error](#) равно [None](#).

Class Result<T>

Namespace: [PhotoCatalog.Domain.Primitives](#)

Assembly: PhotoCatalog.Domain.dll

Представляет результат выполнения операции, возвращающей значение типа **T**.

```
public class Result<T>
```

Type Parameters

T

Тип возвращаемого значения.

Inheritance

[object](#)  ← Result<T>

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Extension Methods

[ResultExtensions.Check<TValue>\(Result<TValue>?, Func<TValue, ResultVoid>\)](#) ,
[ResultExtensions.Check<TValue, TOther>\(Result<TValue>?, Func<TValue, Result<TOther>>\)](#) ,
[ResultExtensions.Ensure<TValue>\(Result<TValue>?, Func<TValue, bool>, Error\)](#) ,
[ResultExtensions.Finally<TValue, TLanding>\(Result<TValue>?, Func<TValue, TLanding>, Func<Error, TLanding>\)](#) ,
[ResultExtensions.OnFailure<TValue>\(Result<TValue>?, Action<Error>\)](#) ,
[ResultExtensions.OnSuccess<TValue>\(Result<TValue>?, Action<TValue>\)](#) ,
[ResultExtensions.ThenTry<TValue>\(Result<TValue>?, Action<TValue>, Func<Exception, Error>\)](#) ,
[ResultExtensions.ThenTry<TValue, TNextValue>\(Result<TValue>?, Func<TValue, TNextValue>, Func<Exception, Error>\)](#) ,
[ResultExtensions.Then<TValue>\(Result<TValue>?, Func<TValue, ResultVoid>\)](#) ,
[ResultExtensions.Then<TValue, TNextValue>\(Result<TValue>?, Func<TValue, Result<TNextValue>>\)](#) ,
[ResultExtensions.ToResult<TValue>\(TValue\)](#) ,

[ResultExtensions.ToResult<TValue>\(TValue?, Error\)](#) ,
[ResultExtensions.Transform<TValue, TNextValue>\(Result<TValue>?, Func<TValue, TNextValue>\)](#).

Remarks

ВНИМАНИЕ: Не создавайте объект через конструктор по умолчанию. Используйте фабрики Success/Failure.

Properties

Error

Объект ошибки. Если операция успешна, содержит [None](#).

```
public Error Error { get; }
```

Property Value

[Error](#)

IsFailure

Указывает, завершилась ли операция с ошибкой.

```
public bool IsFailure { get; }
```

Property Value

[bool](#)

IsSuccess

Указывает, завершилась ли операция успешно.

```
public bool IsSuccess { get; }
```

Property Value

[bool](#)

Value

Возвращает результат операции. Если операция завершилась провалом ([IsFailure](#) равно true), возвращает значение по умолчанию (default/null).

```
public T? Value { get; }
```

Property Value

T

Methods

Deconstruct(out bool, out T?, out Error)

Деконструирует объект результата для удобного использования в паттерн-матчинге и кортежах.

```
public void Deconstruct(out bool isSuccess, out T? value, out Error error)
```

Parameters

isSuccess [bool](#)

Флаг успешности.

value T

Значение (или default при ошибке).

error [Error](#)

Объект ошибки.

Failure(Error)

Создает провальный результат с указанной ошибкой и значением по умолчанию.

```
public static Result<T> Failure(Error error)
```

Parameters

error [Error](#)

Returns

[Result](#)<T>

Success(T)

Создает успешный результат с переданным значением. Гарантирует функциональную безопасность: при передаче null возвращает системную ошибку.

```
public static Result<T> Success(T value)
```

Parameters

value T

Значение успешной операции.

Returns

[Result](#)<T>

Экземпляр [Result<T>](#) со статусом успеха или провала (если передан null).

Operators

implicit operator ResultVoid(Result<T>)

Неявно преобразует Result{T} обобщенного типа в базовый ResultVoid. Обеспечивает статическую диспетчеризацию (полиморфизм) на этапе компиляции.

```
public static implicit operator ResultVoid(Result<T> result)
```

Parameters

result [Result](#)<T>

Returns

[ResultVoid](#)

Class SystemErrors

Namespace: [PhotoCatalog.Domain.Primitives](#)

Assembly: PhotoCatalog.Domain.dll

Реестр критических системных ошибок, которые не связаны с бизнес-правилами, а возникают из-за сбоев в потоке выполнения или инфраструктуре.

```
public static class SystemErrors
```

Inheritance

[object](#)  ← SystemErrors

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  ,
[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  ,
[object.ToString\(\)](#) 

Fields

NullResult

Ошибка, возникающая при получении null значение из объекта обернутого Result.

```
public static readonly Error NullResult
```

Field Value

[Error](#)

NullValue

Ошибка, возникающая при попытке инициализировать успешный результат пустой ссылкой.

```
public static readonly Error NullValue
```

Field Value

[Error](#)

Namespace PhotoCatalog.Domain.Value Objects

Classes

[Dimensions](#)

Value Object для хранения габаритов изображения (ширина × высота в пикселях).
Гарантирует инварианты: ширина и высота строго положительные числа (> 0).

Class Dimensions

Namespace: [PhotoCatalog.Domain.ValueObjects](#)

Assembly: PhotoCatalog.Domain.dll

Value Object для хранения габаритов изображения (ширина × высота в пикселях).
Гарантирует инварианты: ширина и высота строго положительные числа (> 0).

```
public record Dimensions : IEquatable<Dimensions>
```

Inheritance

[object](#)  ← Dimensions

Implements

[IEquatable](#)  <[Dimensions](#)>

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  ,
[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  ,
[object.ToString\(\)](#) 

Extension Methods

[ResultExtensions.ToResult<TValue>\(TValue\)](#) ,
[ResultExtensions.ToResult<TValue>\(TValue?, Error\)](#)

Examples

```
var dimensions = Dimensions.Create(1920, 1080);  
if (dimensions.IsSuccess)  
{  
    Console.WriteLine($"Размер:  
{dimensions.Value.Width}x{dimensions.Value.Height}");  
}
```

Remarks

Dimensions представляет собой неизменяемый объект без идентификатора. Два объекта считаются равными, если имеют одинаковые Width и Height (структурное равенство).

Инварианты:

- Width > 0
- Height > 0

Создание возможно только через [Create\(int, int\)](#) с валидацией.

Properties

Height

Высота изображения в пикселях.

```
public int Height { get; init; }
```

Property Value

[int](#)

Remarks

Допустимые значения: больше 0.

Width

Ширина изображения в пикселях.

```
public int Width { get; init; }
```

Property Value

[int](#)

Remarks

Допустимые значения: больше 0.

Methods

Create(int, int)

Создает объект Dimensions с валидацией инвариантов.

```
public static Result<Dimensions> Create(int width, int height)
```

Parameters

width [int](#)

Ширина в пикселях (больше 0)

height [int](#)

Высота в пикселях (больше 0)

Returns

[Result<Dimensions>](#)

[Success\(T\)](#) с валидными габаритами, или [Failure\(Error\)](#) с [Invalid](#).

Remarks

Оба измерения должны быть больше 0. Если хотя бы одно измерение невалидно, возвращается ошибка.

Namespace PhotoCatalog.Infrastructure. Errors

Classes

[InfrastructureErrors](#)

Единый статический класс (реестр), который содержит все ошибки уровня Infrastructure в виде заранее определенных структур [Error](#).

[InfrastructureErrors.Database](#)

Ошибки, связанные с работой базы данных.

[InfrastructureErrors.FileStorage](#)

Ошибки, связанные с файловым хранилищем.

Class InfrastructureErrors

Namespace: [PhotoCatalog.Infrastructure.Errors](#)

Assembly: PhotoCatalog.Infrastructure.dll

Единый статический класс (реестр), который содержит все ошибки уровня Infrastructure в виде заранее определенных структур [Error](#).

```
public static class InfrastructureErrors
```

Inheritance

[object](#)  ← InfrastructureErrors

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  ,
[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  ,
[object.ToString\(\)](#) 

Remarks

В отличие от [DomainErrors](#) (бизнес-логика) и ApplicationErrors (ошибки потока/поиска), этот реестр содержит ошибки, связанные с физическими сбоями систем (базы данных, диска, сети).

Class InfrastructureErrors.Database

Namespace: [PhotoCatalog.Infrastructure.Errors](#)

Assembly: PhotoCatalog.Infrastructure.dll

Ошибки, связанные с работой базы данных.

```
public static class InfrastructureErrors.Database
```

Inheritance

[object](#)  ← InfrastructureErrors.Database

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  ,
[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  ,
[object.ToString\(\)](#) 

Fields

ConnectionFailed

Ошибка, когда не удалось установить соединение с базой данных.

```
public static readonly Error ConnectionFailed
```

Field Value

[Error](#)

ConstraintViolation

Ошибка, когда нарушена целостность данных (например, дублирование уникального ключа, нарушение внешнего ключа).

```
public static readonly Error ConstraintViolation
```

Field Value

[Error](#)

Class InfrastructureErrors.FileStorage

Namespace: [PhotoCatalog.Infrastructure.Errors](#)

Assembly: PhotoCatalog.Infrastructure.dll

Ошибки, связанные с файловым хранилищем.

```
public static class InfrastructureErrors.FileStorage
```

Inheritance

[object](#)  ← InfrastructureErrors.FileStorage

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  ,
[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  ,
[object.ToString\(\)](#) 

Fields

AccessDenied

Ошибка, когда отказано в доступе к файловой системе.

```
public static readonly Error AccessDenied
```

Field Value

[Error](#)

DiskFull

Ошибка, когда на диске недостаточно свободного места.

```
public static readonly Error DiskFull
```

Field Value

[Error](#)

IOError

Ошибка, когда произошла ошибка ввода-вывода при работе с файлом.

```
public static readonly Error IOError
```

Field Value

[Error](#)